Retrieval-Augmented Generation (RAG)

Background

- LLMs are powerful, but they have 3 structural gaps:
    - Knowledge cutoff: They do not know events after training
    - No access to private data: Your PDFs, your customer databases, …
    - Hallucination: They may make up information when they do not have enough evidence
- Fine-tuning can teach new knowledge, but it is slow, expensive, hard to audit and ages within weeks
- RAG addresses this by injecting evidence into the prompt at query time, rather than encoding it in the model weights

RAG pipeline

- **Indexing (offline)**
  Documents → Chunks → Embeddings → Vector store
- **Query (online)**
  User question → Embed query → Retrieve top-k relevant chunks from vector store → Build prompt with retrieved chunks → LLM generates answer
  ⇒ *This way, the model sees the relevant evidence before it has a chance to make something up*

Embedding

- An embedding is a dense vector that captures meaning
    - Produced by a Transformer encoder
    - Cosine similarity in vector space ≈ semantic similarity in text
- Advantage over keyword-based search
    - Semantic retrieval can identify conceptually relevant documents even when there is little or no lexical overlap
    - For example, the query “How do I cancel my subscription?” may match the document “Account termination procedure” because both express the same underlying intent

Chunking

- A single vector cannot adequately represent all topics, details and contexts in a long document
- Key design choices in RAG:
    - Chunk size: usually 200 - 800 tokens (1 English word is about 1.3 tokens)
    - Chunk overlap: Repeat nearby text to keep context
    - Chunk boundaries: Split a natural text structures
    - Metadata: Add source detail to each chunk (like document name, page number, section title, date and author, …)
    - Hierarchy: Link chunks back to their parent section / document

Retrieval in Production RAG

Vector search alone is often not enough. Real-world RAG systems usually combine several retrieval techniques

- **Hybrid search:** Semantic vectors + Keyword search
    - The user asks: “What is the late penalty for COS30082?”
    - RAG does:
        - Vector search finds chunks about assessment rules
        - Keyword search finds the exact term “COS30082” and “late penalty”
        - The system combines both results
- **Reranking:** Retrieve many chunks, then rerank the best ones
    - The user asks: “When is the project due?”
    - RAG does:
        - First retrieves 50 possible chunks about assessments, schedule, project
        - A reranker checks which chunks best answer the question
        - It keeps the top 5 most relevant chunks for the LLM
- **Query rewriting**: Clarify the user query before search
    - The user asks: “When do we submit it?”
    - RAG does:
        - Rewrites the query as “When is the COS30082 project submission due?”
        - Then it searches using the clearer question
- **Metadata filtering:** Search only within selected users, dates or document types
    - The user asks: “What was the policy in 2026?”
    - RAF does:
        - Searches only documents where year = 2026
        - Or searches only document type = policy
        - This avoids using old or irrelevant documents

Agentic AI

Chatbot versus Agent

- A chatbot is a model that turns text into text
- An agent is a model that takes actions in the world and observes the results
- The model becomes the controller
- The intelligence is in deciding ***What to do next*** given partial information

Agentic AI roles

- Agentic AI = LLM reasoning + tool use + workflow control
    - **LLM for thinking**: Understand, reason, plan, decide
    - **Tools for action**: Search, code, files, databases, APIs
    - **Agent for coordination**: Plan → Act → Observe → Reflect → Continue
- What Agents offer:
    - Planning: Break complex tasks into steps
    - Tool use: Lets AI access data or external tools
    - Memory: Keep useful context across steps
    - Reflection: Check results and retry if needed
    - Automation: Complete multi-step tasks
- Where Agents already work in production
    - Coding agents
      (Claude code, GitHub Copilot, Codex)
    - Browser / computer-use agents
      (Anthropic Computer Use, OpenAI Operator, Google Project Mariner)
    - Operations and support
      (Automated ticket triage, refunds, scheduling

Multi-agent systems

Multi-agent: An AI system where multiple specialised agents work together to complete a task

For example, a multi-agent system where some agents cooperate to produce a research report

- Planner Agent: Breaks the task into steps
- Research Agent: Searches and retrieves evidence
- Analysis Agent: Summarises and checks findings
- Writer Agent: Produces the final report

Multimodal ML

- Multimodel ML combines multiple types of data in 1 system, it learns relationships across different data types to improve understanding and prediction
- Common modalities include:
    - Images
    - Text
    - Audio
    - Video
    - Sensor data
- Multimodal AI is useful when 1 data type alone is incomplete, but **combining modalities can improve context and reduce ambiguity**

Typical Multimodal AI Pipeline

1. Input modalities
   (text, image, audio, video)
2. Feature Extraction / Embeddings
   (CNN, Transformer, audio encoder, …)
3. Fusion
   (Combine information from multiple modalities)
4. Prediction / Generation
   (Classification, captioning, Q&A, recommendation, report generation, …)

Common Multimodal Applications

- **Image captioning**: Image → Text description
  Example: “This photo contains a lion”
- **Visual question answering**: Image + Question → Answer
  Example: “How many cars are in this image?”
- **Autonomous driving**: Cameras + Sensors + Maps → Driving decisions
  Example: Detect pedestrians and plan route and movement
- **Medical Diagnosis**: Scan / Image + Clinical notes → Diagnosis support
  Example: X-ray + Patient history → Make a diagnosis
- **Video Understanding**: Video frame + Audio + Text → Event understanding
  Example: Summarising a lecture video